

```
In [44]: import matplotlib
import matplotlib.pyplot as plt
import os

output_dir = "/Users/cordelia/Documents/PROJECT_12/slides_img"
os.makedirs(output_dir, exist_ok=True)

# Remplace la fonction show() pour sauvegarder automatiquement
original_show = plt.show
counter = [0]

def custom_show():
    counter[0] += 1
    plt.savefig(f"{output_dir}/fig_{counter[0]:02d}.png", dpi=150, bbox_inches='tight')
    original_show()

plt.show = custom_show
print(f"✅ Les graphiques seront sauvegardés dans {output_dir}")
```

✅ Les graphiques seront sauvegardés dans /Users/cordelia/Documents/PROJECT_12/slides_img

1) Chargement des données

Allez, cadeau 🎁 :

```
In [45]: import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

DATA_PATH = "/Users/cordelia/Documents/PROJECT_12/billets.csv"

if not os.path.exists(DATA_PATH):
    print(f"❌ Attention : le fichier '{DATA_PATH}' n'existe pas.")

try:
    df = pd.read_csv(DATA_PATH, sep=";")
    print(f"✅ Données chargées :", df.shape)
    display(df.head())
except Exception as e:
    print(f"❌ Erreur lors du chargement :", e)
    print(f"⚠️ Tu as bien lu le code avant de l'exécuter ? 🤔")
```

✅ Données chargées : (1500, 7)

	is_genuine	diagonal	height_left	height_right	margin_low	margin_up	length
0	True	171.81	104.86	104.95	4.52	2.89	112.83
1	True	171.46	103.36	103.66	3.77	2.99	113.09
2	True	172.69	104.48	103.50	4.40	2.94	113.16
3	True	171.36	103.91	103.94	3.62	3.01	113.51
4	True	171.73	104.28	103.46	4.04	3.48	112.54

2) Analyse exploratoire (EDA)

Là, c'est vous les pro 🙌 , je vous fais entièrement confiance 😊 !

Pssst (indice) : trouvez des variables corrélées à votre objectif, ça marche mieux 😊 .

```
In [46]: # 2.1 Aperçu général
print("=== SHAPE ===")
print(df.shape)

print("\n=== TYPES ===")
print(df.dtypes)

print("\n=== APERÇU ===")
display(df.head(10))
```

```
=== SHAPE ===
(1500, 7)
```

```
=== TYPES ===
is_genuine      bool
diagonal        float64
height_left     float64
height_right    float64
margin_low      float64
margin_up       float64
length          float64
dtype: object
```

```
=== APERÇU ===
```

	is_genuine	diagonal	height_left	height_right	margin_low	margin_up	length
0	True	171.81	104.86	104.95	4.52	2.89	112.83
1	True	171.46	103.36	103.66	3.77	2.99	113.09
2	True	172.69	104.48	103.50	4.40	2.94	113.16
3	True	171.36	103.91	103.94	3.62	3.01	113.51
4	True	171.73	104.28	103.46	4.04	3.48	112.54
5	True	172.17	103.74	104.08	4.42	2.95	112.81
6	True	172.34	104.18	103.85	4.58	3.26	112.81
7	True	171.88	103.76	104.08	3.98	2.92	113.08
8	True	172.47	103.92	103.67	4.00	3.25	112.85
9	True	172.47	104.07	104.02	4.04	3.25	113.45

```
In [47]: # 2.2 Valeurs manquantes
print("=== VALEURS MANQUANTES ===")
print(df.isnull().sum())
```

```

=== VALEURS MANQUANTES ===
is_genuine      0
diagonal        0
height_left     0
height_right    0
margin_low      37
margin_up       0
length          0
dtype: int64

```

```

In [48]: # Répartition des NaN de margin_low par classe
print(df[df["margin_low"].isnull()]["is_genuine"].value_counts())

```

```

is_genuine
True      29
False     8
Name: count, dtype: int64

```

```

In [49]: # Doublons
print("=== DOUBLONS ===")
print(df.duplicated().sum())

```

```

=== DOUBLONS ===
0

```

```

In [50]: # Distribution de margin_low
print(df["margin_low"].describe())
print(f"\nSkewness : {df['margin_low'].skew():.3f}")

```

```

count      1463.000000
mean        4.485967
std         0.663813
min         2.980000
25%         4.015000
50%         4.310000
75%         4.870000
max         6.900000
Name: margin_low, dtype: float64

```

```

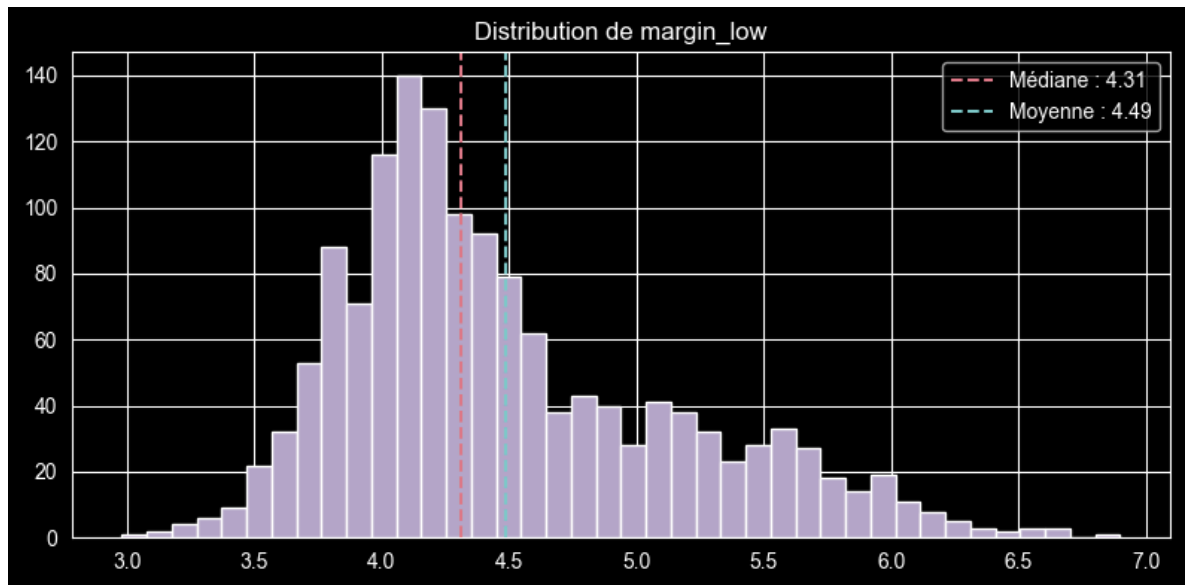
Skewness : 0.863

```

```

In [51]: # Distribution de margin_low
plt.figure(figsize=(8, 4))
plt.hist(df["margin_low"].dropna(), bins=40, color="#b5a9c9", edgecolor="")
plt.axvline(df["margin_low"].median(), color="#e07b8a", linestyle="--", label="Median")
plt.axvline(df["margin_low"].mean(), color="#7ec8c8", linestyle="--", label="Mean")
plt.title("Distribution de margin_low")
plt.legend()
plt.tight_layout()
plt.show()

```



```
In [52]: # 2.3 bis - Imputation par Random Forest
from sklearn.ensemble import RandomForestRegressor

# Séparer les lignes avec et sans NaN (sans is_genuine = pas de data leak
features_imp = ["diagonal", "height_left", "height_right", "margin_up", "

df_known = df[df["margin_low"].notna()]
df_missing = df[df["margin_low"].isna()]

X_train_imp = df_known[features_imp]
y_train_imp = df_known["margin_low"]
X_pred_imp = df_missing[features_imp]

# Entraînement
rf_imp = RandomForestRegressor(random_state=42)
rf_imp.fit(X_train_imp, y_train_imp)

# Imputation
df.loc[df["margin_low"].isna(), "margin_low"] = rf_imp.predict(X_pred_imp

print("Valeurs manquantes après imputation :")
print(df["margin_low"].isnull().sum())
```

Valeurs manquantes après imputation :
0

```
In [53]: df_original = pd.read_csv("/Users/cordelia/Documents/PROJECT_12/billets.c
idx_imputed = df_original[df_original["margin_low"].isnull()].index
display(df.loc[idx_imputed])
```

	is_genuine	diagonal	height_left	height_right	margin_low	margin_up	length
72	True	171.94	103.89	103.45	3.9337	3.25	112.79
99	True	171.93	104.07	104.18	4.1371	3.14	113.08
151	True	172.07	103.80	104.38	4.0588	3.02	112.95
197	True	171.45	103.66	103.80	4.0572	3.62	113.27
241	True	171.83	104.14	104.06	5.3095	3.02	112.30
251	True	171.80	103.26	102.82	4.2286	2.95	113.25
284	True	171.92	103.83	103.76	4.0044	3.23	113.29
334	True	171.85	103.70	103.96	4.1357	3.00	113.30
410	True	172.56	103.72	103.51	3.9892	3.12	112.95
413	True	172.30	103.66	103.50	3.9428	3.16	112.95
445	True	172.34	104.42	103.22	4.0874	3.01	112.95
481	True	171.81	103.53	103.96	4.1124	2.71	113.95
505	True	172.01	103.97	104.05	4.2001	2.98	113.65
611	True	171.80	103.68	103.49	4.0906	3.30	112.84
654	True	171.97	103.69	103.54	4.0183	2.70	112.79
675	True	171.60	103.85	103.91	4.1433	2.56	113.27
710	True	172.03	103.97	103.86	4.1320	3.07	112.65
739	True	172.07	103.74	103.76	4.2358	3.09	112.44
742	True	172.14	104.06	103.96	4.1464	3.24	113.07
780	True	172.41	103.95	103.79	4.0942	3.13	113.44
798	True	171.96	103.84	103.62	4.4017	3.01	114.44
844	True	171.62	104.14	104.49	4.2948	2.99	113.35
845	True	172.02	104.21	104.05	4.3051	2.90	113.65
871	True	171.37	104.07	103.75	4.0228	3.07	113.27
895	True	171.81	103.68	103.80	4.1375	2.98	113.85
919	True	171.92	103.68	103.45	4.2959	2.58	113.65
945	True	172.09	103.74	103.52	4.0538	3.02	112.75
946	True	171.63	103.87	104.66	4.4634	3.27	112.65
981	True	172.02	104.23	103.72	4.1048	2.99	113.35
1076	False	171.57	104.27	104.44	5.3058	3.21	111.85
1121	False	171.40	104.38	104.19	4.9868	3.17	112.35
1176	False	171.59	104.05	103.94	5.4322	3.02	111.25
1303	False	172.17	104.49	103.76	5.4960	2.93	111.25
1315	False	172.08	104.15	104.17	5.2953	3.40	112.25

	is_genuine	diagonal	height_left	height_right	margin_low	margin_up	length
1347	False	171.72	104.46	104.12	5.0734	3.61	110.3
1435	False	172.66	104.33	104.41	4.8617	3.56	111.4
1438	False	171.90	104.28	104.29	5.3820	3.24	111.4

```
In [54]: # 2.4 Répartition des classes
print(df["is_genuine"].value_counts())
print(df["is_genuine"].value_counts(normalize=True).mul(100).round(1).ast
```

```
is_genuine
True      1000
False      500
Name: count, dtype: int64
is_genuine
True      66.7 %
False     33.3 %
Name: proportion, dtype: str
```

```
In [55]: # 2.4 Statistiques descriptives par classe
df.groupby("is_genuine").describe().round(2)
```

```
Out [55]:
```

		count	mean	std	min	25%	50%	75%	max	count	me
is_genuine											
	False	500.0	171.90	0.31	171.04	171.69	171.91	172.09	173.01	500.0	104
	True	1000.0	171.99	0.30	171.04	171.79	171.99	172.20	172.92	1000.0	103.

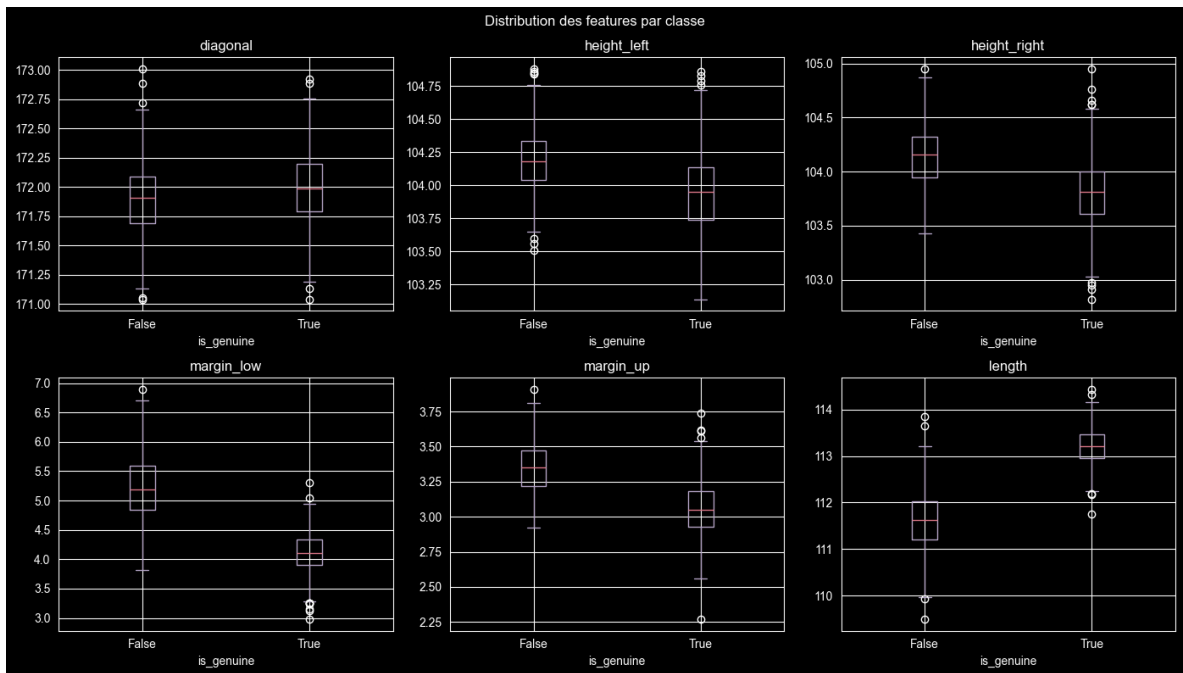
2 rows × 48 columns

```
In [56]: # 2.5 Distribution des features par classe
features = ["diagonal", "height_left", "height_right", "margin_low", "mar

fig, axes = plt.subplots(2, 3, figsize=(14, 8))
axes = axes.flatten()

for i, feature in enumerate(features):
    df.boxplot(column=feature, by="is_genuine", ax=axes[i],
                boxprops=dict(color="#b5a9c9"),
                medianprops=dict(color="#e07b8a"),
                whiskerprops=dict(color="#b5a9c9"),
                capprops=dict(color="#b5a9c9"))
    axes[i].set_title(feature)
    axes[i].set_xlabel("is_genuine")

plt.suptitle("Distribution des features par classe")
plt.tight_layout()
plt.show()
```



```
In [57]: # 2.5 bis – Bar charts par classe
features = ["diagonal", "height_left", "height_right", "margin_low", "margin_up", "length"]

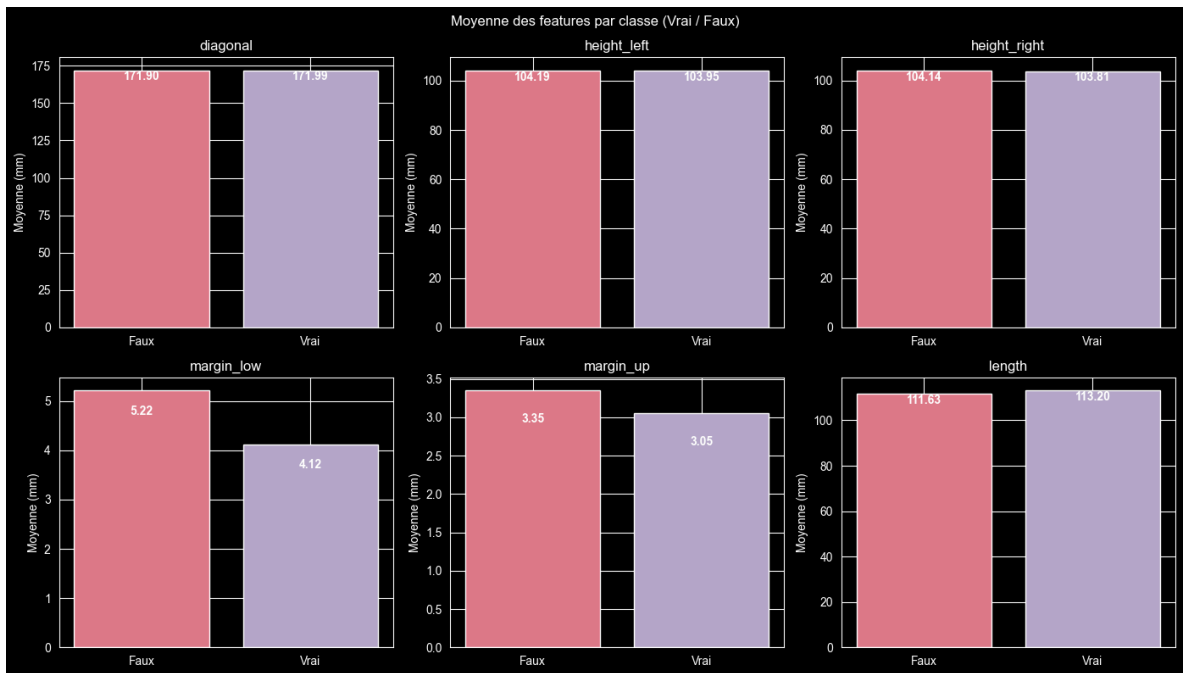
fig, axes = plt.subplots(2, 3, figsize=(14, 8))
axes = axes.flatten()

means = df.groupby("is_genuine")[features].mean()

for i, feature in enumerate(features):
    bars = axes[i].bar(["Faux", "Vrai"],
                       [means.loc[False, feature], means.loc[True, feature]],
                       color=["#e07b8a", "#b5a9c9"],
                       edgecolor="white")
    axes[i].set_title(feature)
    axes[i].set_ylabel("Moyenne (mm)")

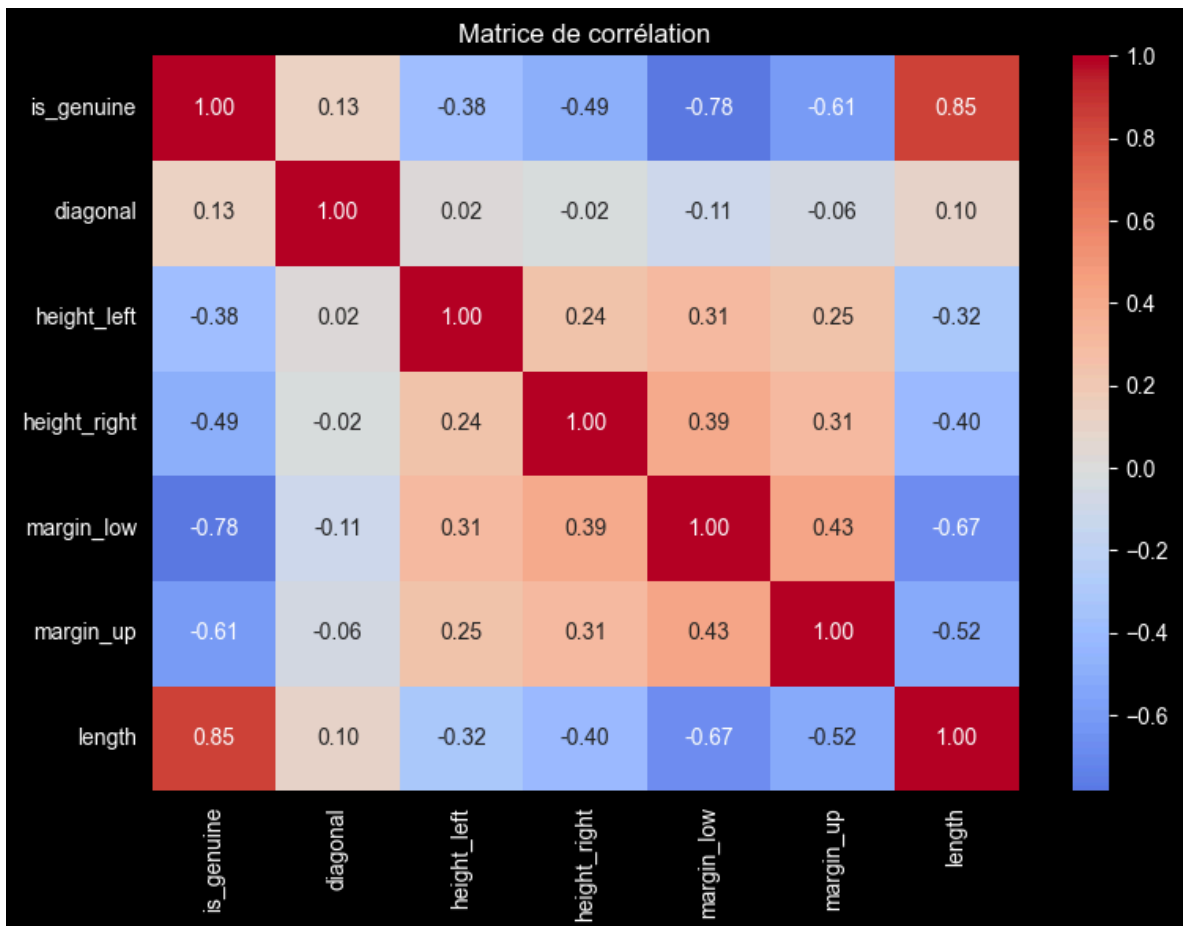
    for bar in bars:
        axes[i].text(bar.get_x() + bar.get_width()/2,
                     bar.get_height() - 0.3,
                     f"{bar.get_height():.2f}",
                     ha="center", va="top", color="white", fontweight="bold")

plt.suptitle("Moyenne des features par classe (Vrai / Faux)")
plt.tight_layout()
plt.show()
```



```
In [58]: # 2.6 Matrice de corrélation
import seaborn as sns

plt.figure(figsize=(8, 6))
corr = df.copy()
corr["is_genuine"] = corr["is_genuine"].astype(int)
sns.heatmap(corr.corr(), annot=True, fmt=".2f", cmap="coolwarm", center=0)
plt.title("Matrice de corrélation")
plt.tight_layout()
plt.show()
```



3) Prétraitement

Le but est de transformer tes données brutes (les valeurs mesurées des billets) en données que les algorithmes de Machine Learning peuvent comprendre efficacement.

D'abord, il faut séparer les variables explicatives et la cible :

X = les mesures du billet (longueur, hauteur, diagonales...)

y = l'étiquette (vrai ou faux)

➡ Pourquoi ? Parce que le modèle doit apprendre à prédire y en fonction de X.

```
In [59]: # 3.1 Séparation features / cible
X = df.drop(columns=["is_genuine"])
y = df["is_genuine"]
```

Autre point important : on entraîne toujours le modèle sur un morceau des données, et on teste sur un autre qu'il n'a jamais vu.

➡ Pourquoi ? Pour éviter le sur-apprentissage (le modèle apprend par cœur sans savoir généraliser).

```
In [60]: # 3.2 Séparation train / test
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,

print(f"Train : {X_train.shape[0]} billets")
print(f"Test  : {X_test.shape[0]} billets")
```

Train : 1200 billets

Test : 300 billets

Enfin, on met toutes les variables sur la même échelle (moyenne = 0, écart-type = 1).

➡ Pourquoi ? Parce que certains modèles, comme le KNN ou la Régression Logistique, sont sensibles aux différences d'échelle. Sans standardisation, la variable la plus grande numériquement prend trop d'importance.

```
In [61]: # 3.3 Normalisation
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

4) Modélisation

Dans tous domaines, vous ne pouvez pas progresser si vous ne comprenez pas comment vos outils fonctionnent.

Avant d'utiliser les modèles mathématiques qui suivent, je vous recommande donc de visionner les vidéos suivantes:

- [StatQuest: Logistic Regression](#)
- [StatQuest: K-nearest neighbors, Clearly Explained](#)
- [StatQuest: Random Forests Part 1 - Building, Using and Evaluating](#)
- [StatQuest: Random Forests Part 2: Missing data and clustering](#)
- [StatQuest: K-means clustering](#)

```
In [62]: from sklearn.linear_model import LogisticRegression

# 1) Initialisez une régression logistique.
# Quels sont les paramètres ajustables de ce modèle ?
# Bien que vous puissiez laisser la majorité d'entre eux par défaut,
# il vaut mieux, à minima, ajuster `max_iter` pour plafonner le temps
from sklearn.linear_model import LogisticRegression
lr = LogisticRegression(max_iter=1000, random_state=42)

# 2) Entraînez le modèle avec vos données X_train et y_train
lr.fit(X_train_scaled, y_train)

# 3) Utiliser la régression logistique pour déterminer les vrais / faux b
y_pred_lr = lr.predict(X_test_scaled)
```

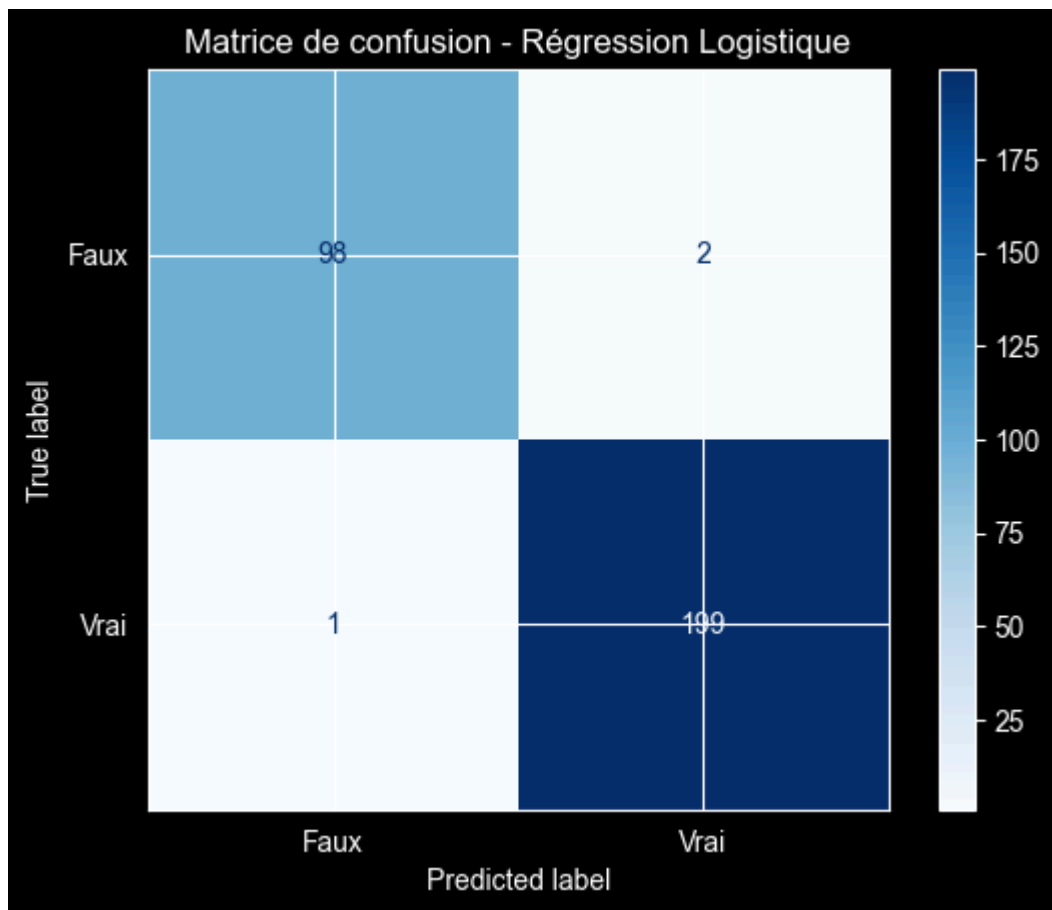
```
In [63]: # 4.1 Résultats - Régression Logistique
from sklearn.metrics import classification_report, confusion_matrix, Conf

print("=== RÉGRESSION LOGISTIQUE ===")
print(classification_report(y_test, y_pred_lr, target_names=["Faux", "Vra

cm_lr = confusion_matrix(y_test, y_pred_lr)
disp = ConfusionMatrixDisplay(confusion_matrix=cm_lr, display_labels=["Fa
disp.plot(cmap="Blues")
plt.title("Matrice de confusion - Régression Logistique")
plt.show()
```

```
=== RÉGRESSION LOGISTIQUE ===
```

	precision	recall	f1-score	support
Faux	0.99	0.98	0.98	100
Vrai	0.99	0.99	0.99	200
accuracy			0.99	300
macro avg	0.99	0.99	0.99	300
weighted avg	0.99	0.99	0.99	300



```
In [64]: from sklearn.neighbors import KNeighborsClassifier

# Répéter les même étapes que pour la régression logistique mais avec le
# 4.2 KNN
from sklearn.neighbors import KNeighborsClassifier

# Paramètres ajustables principaux :
# - n_neighbors : nombre de voisins à considérer - défaut 5
# - metric : distance utilisée ('euclidean', 'manhattan'...) - défaut 'mi
# - weights : pondération des voisins ('uniform', 'distance') - défaut 'u

# 1) Initialisation
knn = KNeighborsClassifier(n_neighbors=11)

# 2) Entraînement
knn.fit(X_train_scaled, y_train)

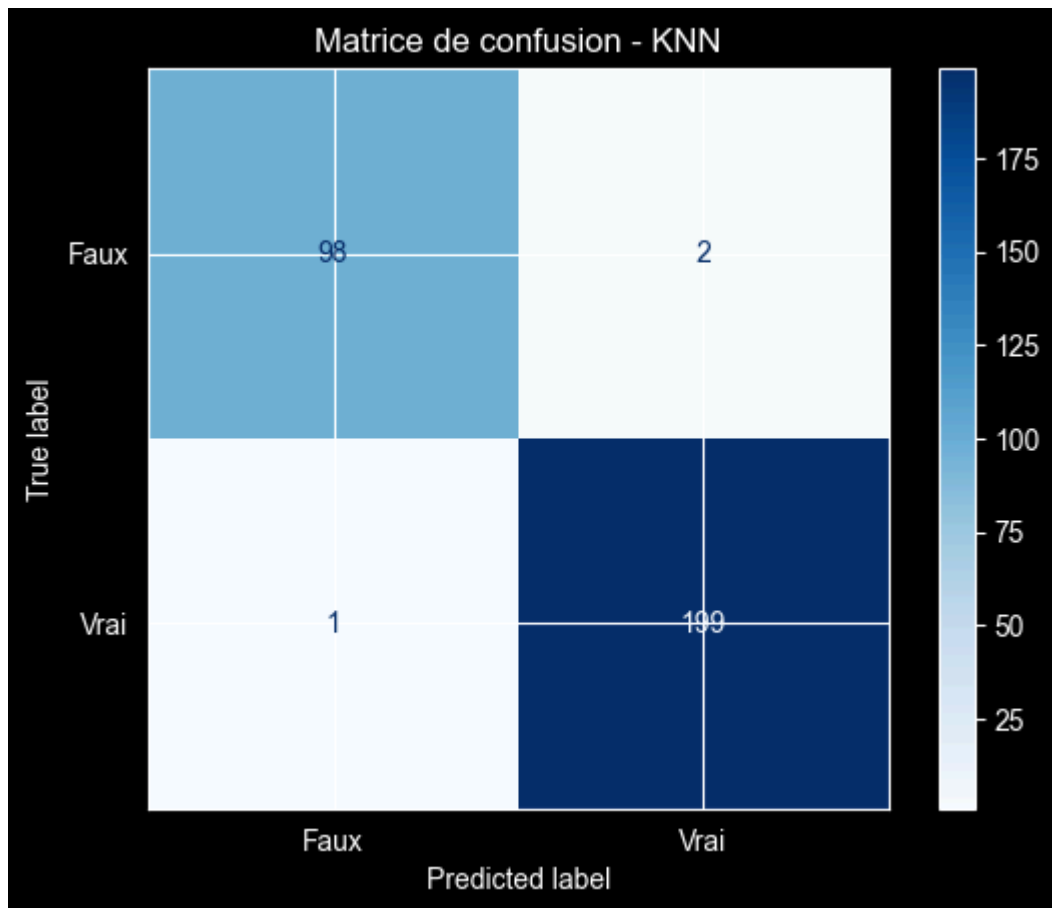
# 3) Prédiction
y_pred_knn = knn.predict(X_test_scaled)

# 4) Résultats
print("=== KNN ===")
print(classification_report(y_test, y_pred_knn, target_names=["Faux", "Vr

cm_knn = confusion_matrix(y_test, y_pred_knn)
disp = ConfusionMatrixDisplay(confusion_matrix=cm_knn, display_labels=["F
disp.plot(cmap="Blues")
plt.title("Matrice de confusion - KNN")
plt.show()
```

=== KNN ===

	precision	recall	f1-score	support
Faux	0.99	0.98	0.98	100
Vrai	0.99	0.99	0.99	200
accuracy			0.99	300
macro avg	0.99	0.99	0.99	300
weighted avg	0.99	0.99	0.99	300



```
In [65]: from sklearn.ensemble import RandomForestClassifier

# 4.3 Random Forest
from sklearn.ensemble import RandomForestClassifier

# Paramètres ajustables principaux :
# - n_estimators : nombre d'arbres – défaut 100
# - max_depth : profondeur maximale des arbres – défaut None
# - min_samples_split : minimum d'échantillons pour splitter un nœud – défaut 2
# - random_state : reproductibilité

# 1) Initialisation
rf = RandomForestClassifier(n_estimators=100, random_state=42)

# 2) Entraînement
rf.fit(X_train_scaled, y_train)

# 3) Prédiction
y_pred_rf = rf.predict(X_test_scaled)

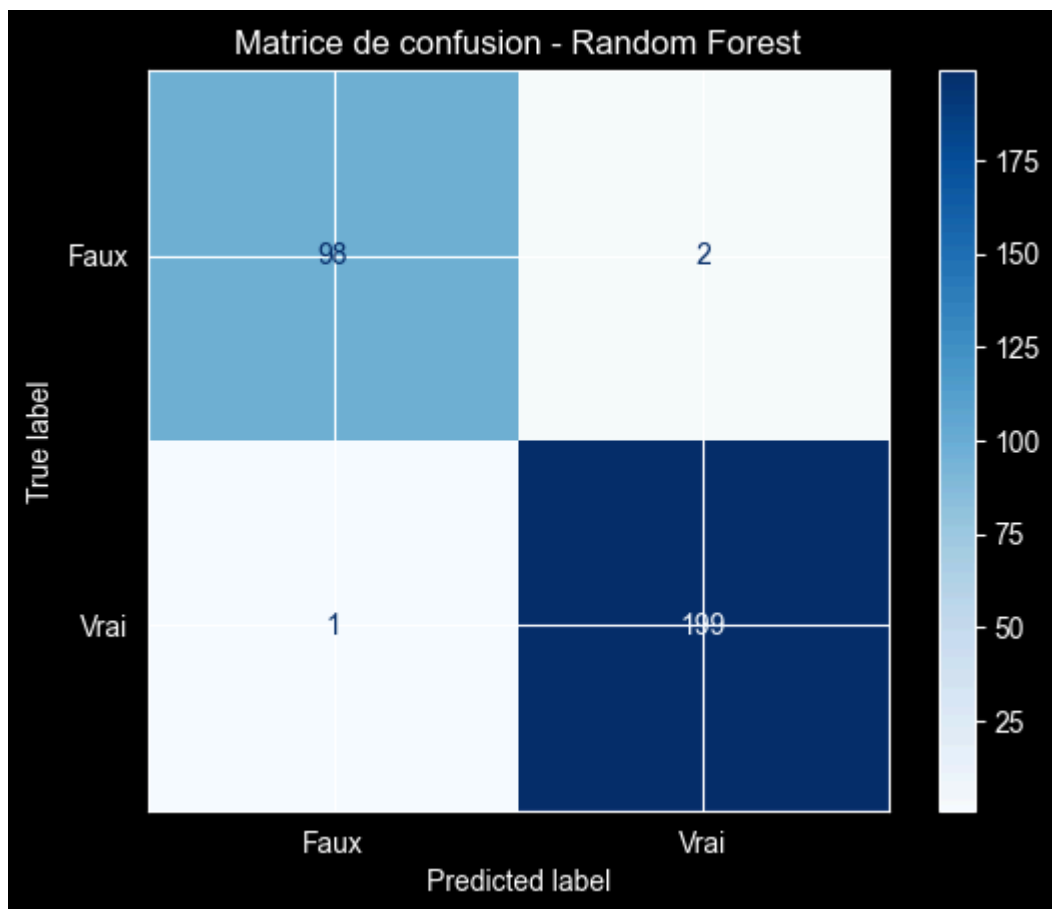
# 4) Résultats
print("=== RANDOM FOREST ===")
```

```
print(classification_report(y_test, y_pred_rf, target_names=["Faux", "Vrai"]

cm_rf = confusion_matrix(y_test, y_pred_rf)
disp = ConfusionMatrixDisplay(confusion_matrix=cm_rf, display_labels=["Fa
disp.plot(cmap="Blues")
plt.title("Matrice de confusion - Random Forest")
plt.show()
```

=== RANDOM FOREST ===

	precision	recall	f1-score	support
Faux	0.99	0.98	0.98	100
Vrai	0.99	0.99	0.99	200
accuracy			0.99	300
macro avg	0.99	0.99	0.99	300
weighted avg	0.99	0.99	0.99	300



```
In [66]: # 4.4 K-Means
from sklearn.cluster import KMeans

# Différences fondamentales avec les modèles précédents :
# - K-Means est un algorithme NON SUPERVISÉ - il n'utilise pas y_train
# - Il ne prédit pas une classe, il regroupe les données en clusters
# - Il ne connaît pas les étiquettes "vrai" / "faux" pendant l'entraînement
# - Les clusters qu'il forme ne correspondent pas forcément à nos classes

# Est-il pertinent ici ?
# Oui et non. Il peut révéler une structure naturelle dans les données
# (est-ce que les billets se séparent naturellement en 2 groupes ?)
# mais il ne peut pas prédire directement vrai/faux comme les autres.
# L'astuce : on utilise les centroïdes pour assigner une classe à chaque
# et on compare avec les vraies étiquettes.
```

```

# 1) Initialisation
kmeans = KMeans(n_clusters=2, random_state=42, n_init=10)

# 2) Entraînement (sans y !)
kmeans.fit(X_train_scaled)

# 3) Prédiction
clusters = kmeans.predict(X_test_scaled)

```

```

In [67]: # Alignement des clusters avec les vraies classes
import numpy as np

# On vérifie quel cluster correspond à quel label
for cluster in [0, 1]:
    mask = clusters == cluster
    genuine_count = y_test[mask].sum()
    print(f"Cluster {cluster} : {genuine_count} vrais / {mask.sum() - gen

```

Cluster 0 : 2 vrais / 98 faux
 Cluster 1 : 198 vrais / 2 faux

```

In [68]: # Mapping clusters → classes
y_pred_kmeans = np.where(clusters == 1, True, False)

print("=== K-MEANS ===")
print(classification_report(y_test, y_pred_kmeans, target_names=["Faux",

cm_km = confusion_matrix(y_test, y_pred_kmeans)
disp = ConfusionMatrixDisplay(confusion_matrix=cm_km, display_labels=["Fa
disp.plot(cmap="Blues")
plt.title("Matrice de confusion - K-Means")
plt.show()

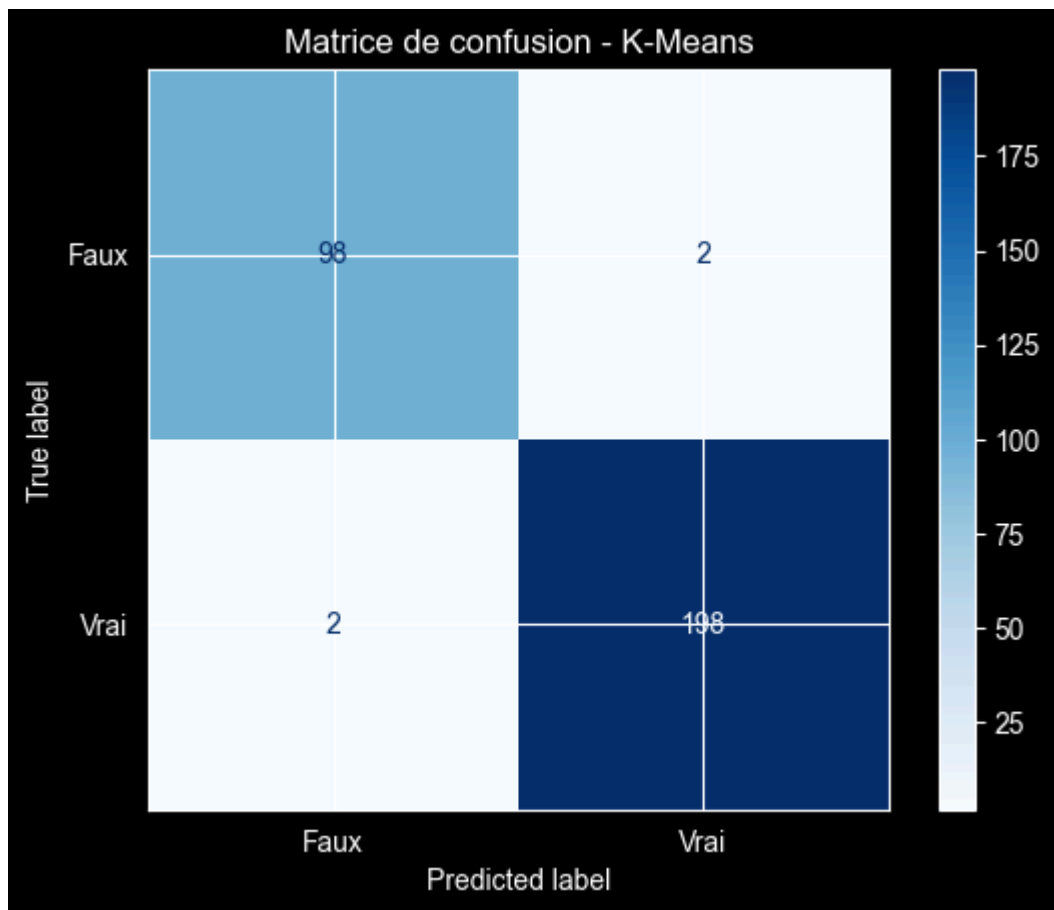
```

```

=== K-MEANS ===

```

	precision	recall	f1-score	support
Faux	0.98	0.98	0.98	100
Vrai	0.99	0.99	0.99	200
accuracy			0.99	300
macro avg	0.98	0.98	0.98	300
weighted avg	0.99	0.99	0.99	300



```
In [69]: # 4.5 GridSearchCV – Régression Logistique
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import make_scorer, recall_score

scorer_faux = make_scorer(recall_score, pos_label=False)

param_grid_lr = {
    "logisticregression__C": [0.001, 0.01, 0.1, 1, 10, 100],
    "logisticregression__solver": ["lbfgs", "saga", "liblinear"],
    "logisticregression__max_iter": [500, 1000]
}

pipeline_lr = make_pipeline(
    StandardScaler(),
    LogisticRegression(random_state=42)
)

grid_lr = GridSearchCV(
    pipeline_lr,
    param_grid_lr,
    cv=5,
    scoring=scorer_faux,
    n_jobs=-1,
    verbose=1
)

grid_lr.fit(X_train, y_train)

print("✅ Meilleurs paramètres :")
```

```
print(grid_lr.best_params_)
print(f"\nMeilleur recall Faux (CV) : {grid_lr.best_score_:.3f}")
```

Fitting 5 folds for each of 36 candidates, totalling 180 fits

✓ Meilleurs paramètres :

```
{'logisticregression__C': 0.001, 'logisticregression__max_iter': 500, 'logisticregression__solver': 'liblinear'}
```

Meilleur recall Faux (CV) : 0.985

```
In [70]: # Test compromis C : recall vs certitude
for C_val in [0.001, 0.005, 0.01, 0.05, 0.1, 0.5, 1]:
    pipe = make_pipeline(StandardScaler(), LogisticRegression(C=C_val, solver='liblinear'))
    pipe.fit(X_train, y_train)
    y_pred = pipe.predict(X_test)
    probas = pipe.predict_proba(X_test)
    recall = recall_score(y_test, y_pred, pos_label=False)
    certitude_moy = probas.max(axis=1).mean()
    print(f"C={C_val:<6} | Recall Faux: {recall:.3f} | Certitude moyenne: {certitude_moy:.3f}")
```

C=0.001	Recall Faux: 1.000	Certitude moyenne: 67.31%
C=0.005	Recall Faux: 0.990	Certitude moyenne: 82.53%
C=0.01	Recall Faux: 0.990	Certitude moyenne: 87.49%
C=0.05	Recall Faux: 0.990	Certitude moyenne: 94.39%
C=0.1	Recall Faux: 0.990	Certitude moyenne: 95.94%
C=0.5	Recall Faux: 0.990	Certitude moyenne: 97.83%
C=1	Recall Faux: 0.980	Certitude moyenne: 98.23%

```
In [71]: # Modèle final avec C=0.1
pipeline_final = make_pipeline(
    StandardScaler(),
    LogisticRegression(C=0.1, solver="liblinear", max_iter=500, random_state=42)
)

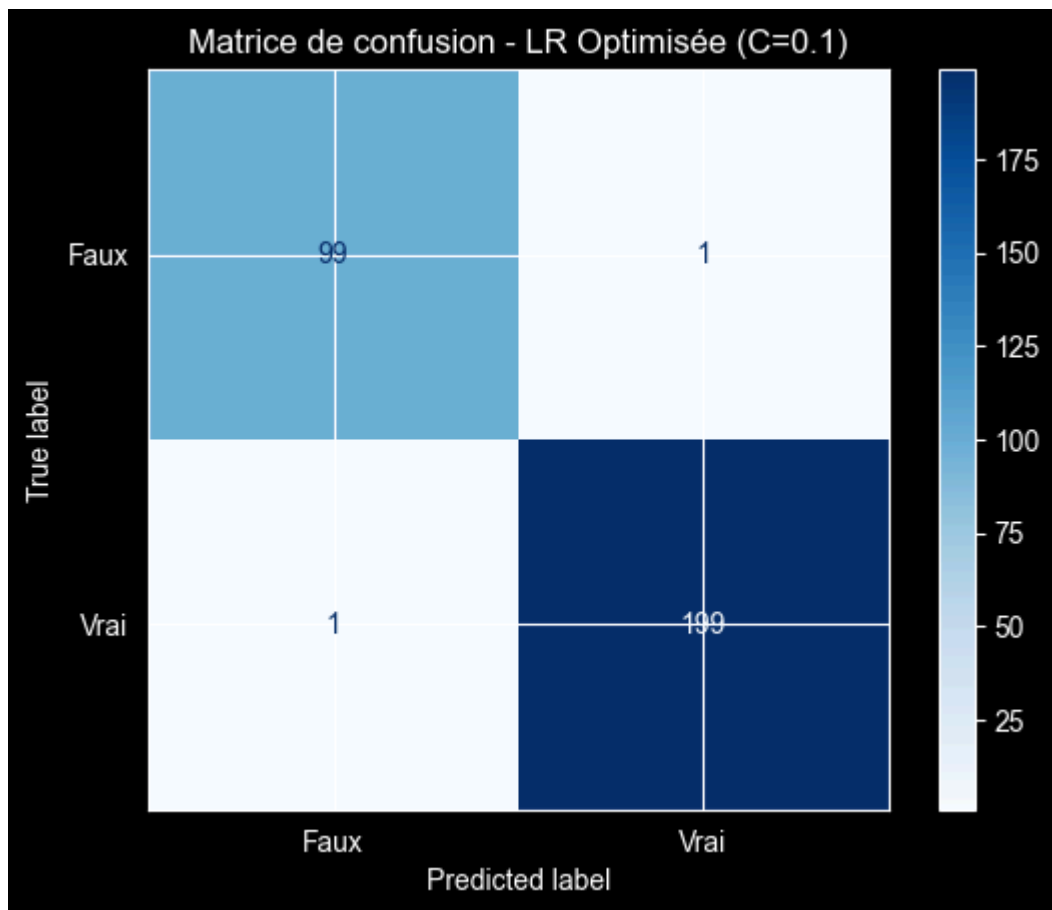
pipeline_final.fit(X_train, y_train)
y_pred_lr_opt = pipeline_final.predict(X_test)

print("=== LR OPTIMISÉE (C=0.1) ===")
print(classification_report(y_test, y_pred_lr_opt, target_names=["Faux", "Vrai"]))

cm = confusion_matrix(y_test, y_pred_lr_opt)
ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["Faux", "Vrai"])
plt.title("Matrice de confusion - LR Optimisée (C=0.1)")
plt.show()
```

```
=== LR OPTIMISÉE (C=0.1) ===
```

	precision	recall	f1-score	support
Faux	0.99	0.99	0.99	100
Vrai	0.99	0.99	0.99	200
accuracy			0.99	300
macro avg	0.99	0.99	0.99	300
weighted avg	0.99	0.99	0.99	300



```
In [72]: # ACP - Projection de la population des billets
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_train_scaled)

print(f"Variance expliquée : {pca.explained_variance_ratio_}")
print(f"Variance totale expliquée : {sum(pca.explained_variance_ratio_):.1%}")

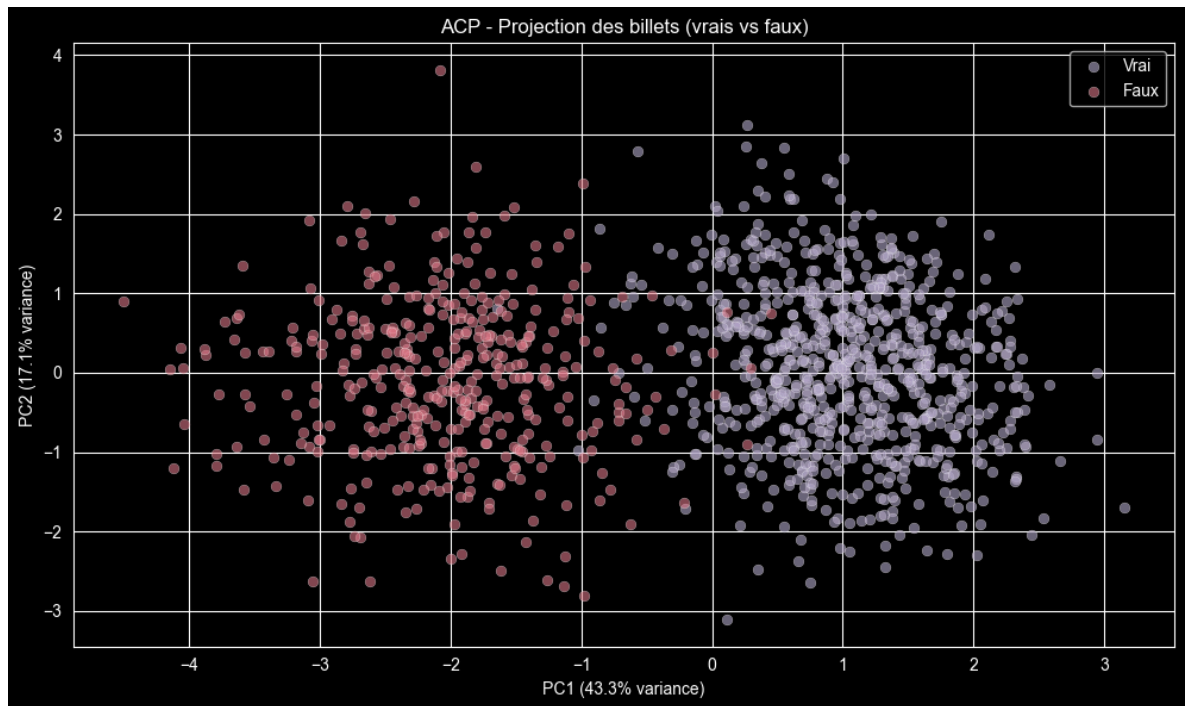
plt.figure(figsize=(10, 6))
colors = {True: "#b5a9c9", False: "#e07b8a"}
labels = {True: "Vrai", False: "Faux"}

for classe in [True, False]:
    mask = y_train == classe
    plt.scatter(X_pca[mask, 0], X_pca[mask, 1],
                c=colors[classe], label=labels[classe],
                alpha=0.6, edgecolors="white", linewidth=0.3, s=40)

plt.xlabel(f"PC1 ({pca.explained_variance_ratio_[0]:.1%} variance)")
plt.ylabel(f"PC2 ({pca.explained_variance_ratio_[1]:.1%} variance)")
plt.title("ACP - Projection des billets (vrais vs faux)")
plt.legend()
plt.tight_layout()
plt.show()
```

Variance expliquée : [0.43275516 0.17066295]

Variance totale expliquée : 60.34%



```
In [73]: # Cross-validation - scores par fold
from sklearn.model_selection import cross_val_score

cv_scores = cross_val_score(
    pipeline_final,
    X_train, y_train,
    cv=5,
    scoring=scorer_faux
)

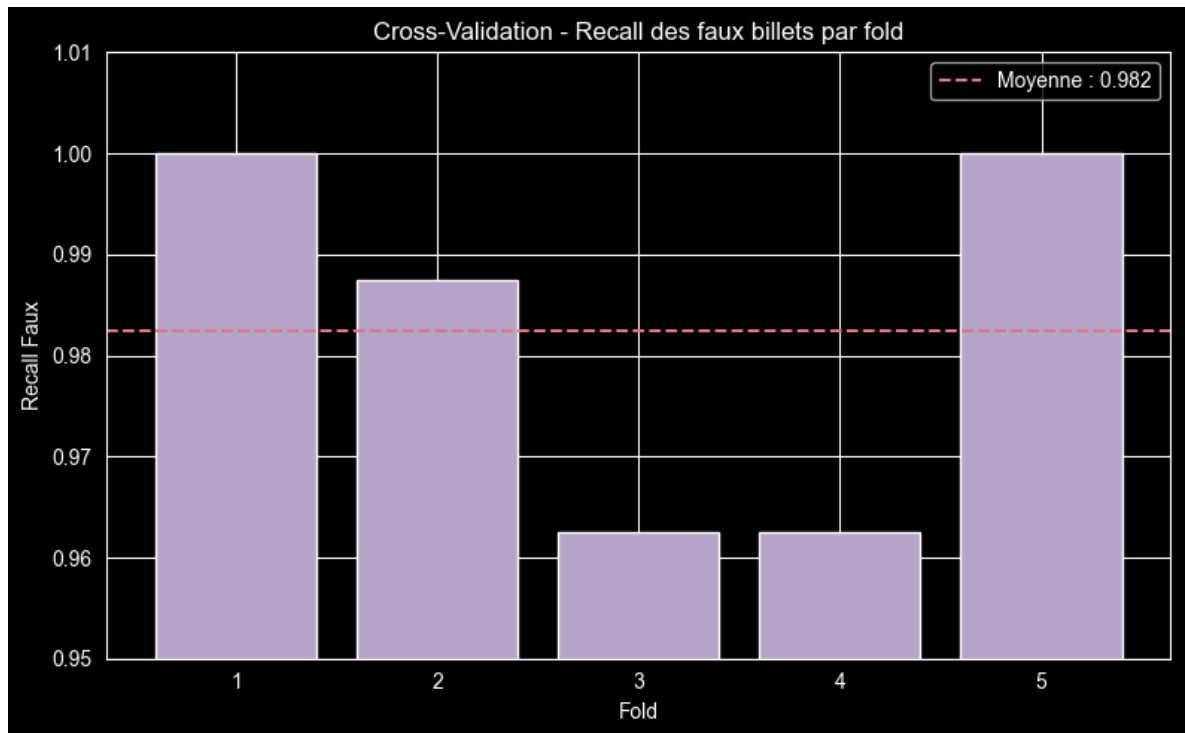
print(f"Scores par fold : {cv_scores.round(3)}")
print(f"Moyenne : {cv_scores.mean():.3f}")
print(f"Écart-type : {cv_scores.std():.3f}")

plt.figure(figsize=(8, 5))
plt.bar(range(1, 6), cv_scores, color="#b5a9c9", edgecolor="white")
plt.axhline(cv_scores.mean(), color="#e07b8a", linestyle="--", label=f"Mo
plt.xlabel("Fold")
plt.ylabel("Recall Faux")
plt.title("Cross-Validation - Recall des faux billets par fold")
plt.ylim(0.95, 1.01)
plt.legend()
plt.tight_layout()
plt.show()
```

Scores par fold : [1. 0.988 0.962 0.962 1.]

Moyenne : 0.982

Écart-type : 0.017



```
In [74]: # Visualisation K-Fold
from sklearn.model_selection import KFold
import numpy as np

kf = KFold(n_splits=5)
fig, axes = plt.subplots(5, 1, figsize=(10, 6))

for i, (train_idx, test_idx) in enumerate(kf.split(X_train)):
    axes[i].barh(0, len(X_train), color="#b5a9c9", height=0.5)
    axes[i].barh(0, len(test_idx), left=test_idx[0], color="#e07b8a", height=0.5)
    axes[i].set_yticks([0])
    axes[i].set_yticklabels([f"Split {i+1}"])
    axes[i].set_xlim(0, len(X_train))
    axes[i].set_xticks([])

from matplotlib.patches import Patch
legend = [Patch(color="#b5a9c9", label="Train Set"), Patch(color="#e07b8a", label="Test Set")]
axes[0].legend(handles=legend, loc="upper right")
plt.suptitle("Cross-Validation - K-Fold (k=5)", fontweight="bold")
plt.tight_layout()
plt.show()
```



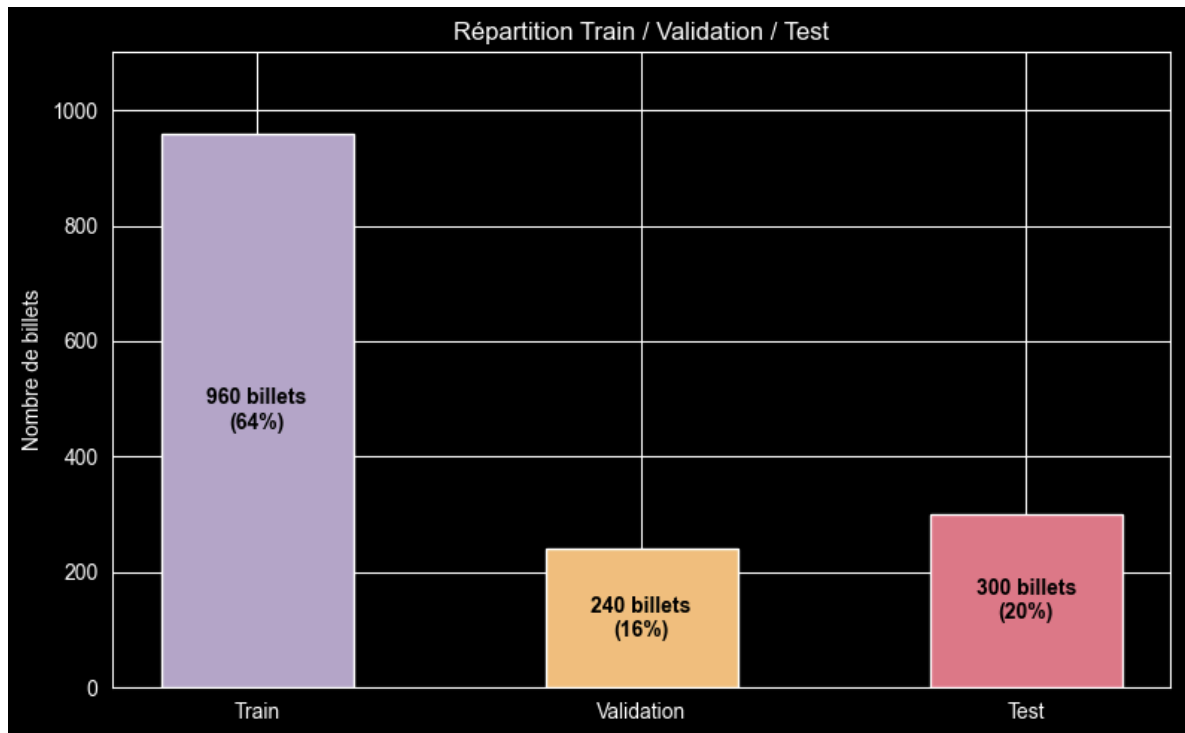
```
In [75]: # Train / Validation / Test split
from sklearn.model_selection import train_test_split

X_train_val, X_test_f, y_train_val, y_test_f = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
X_train_f, X_val_f, y_train_f, y_val_f = train_test_split(
    X_train_val, y_train_val, test_size=0.2, random_state=42, stratify=y
)

print(f"Train      : {X_train_f.shape[0]} billets ({X_train_f.shape[0]/len(X):.0f}%)")
print(f"Validation : {X_val_f.shape[0]} billets ({X_val_f.shape[0]/len(X):.0f}%)")
print(f"Test       : {X_test_f.shape[0]} billets ({X_test_f.shape[0]/len(X):.0f}%)")

plt.figure(figsize=(8, 5))
tailles = [X_train_f.shape[0], X_val_f.shape[0], X_test_f.shape[0]]
labels = ["Train", "Validation", "Test"]
couleurs = ["#b5a9c9", "#f0c27f", "#e07b8a"]
bars = plt.bar(labels, tailles, color=couleurs, edgecolor="white", width=0.8)
for bar, val in zip(bars, tailles):
    pct = val / len(X) * 100
    plt.text(bar.get_x() + bar.get_width()/2,
             bar.get_height() / 2,
             f"{val} billets\n({pct:.0f}%)",
             ha="center", va="center", fontweight="bold", color="black")
plt.title("Répartition Train / Validation / Test")
plt.ylabel("Nombre de billets")
plt.ylim(0, 1100)
plt.tight_layout()
plt.show()
```

```
Train      : 960 billets (64%)
Validation : 240 billets (16%)
Test       : 300 billets (20%)
```



Sélection du meilleur modèle

Mais alors, quel est le meilleur modèle pour résoudre ce problème ?

Pour répondre à cette question, il vous faut bien comprendre l'intérêt de chacune des métriques d'évaluation qui existent pour ce type de problème.

Vous me voyez venir... 😎 : StatQuest !!! 🤖

- [Machine Learning Fundamentals: Sensitivity and Specificity](#)
- [Machine Learning Fundamentals: The Confusion Matrix](#)
- [ROC and AUC, Clearly Explained!](#)

Vous avez aussi des petits modules sympatiques sur Sklearn qui vous machent grandement le travail:

- [Confusion Matrix](#)
- [Confusion Matrix Display](#)
- [Classification Report](#)

A vous de jouer ! 🚀

```
In [76]: from sklearn.metrics import accuracy_score, precision_score, recall_score  
  
# C'est là qu'on voit qui sont les bons 😊!
```

```
In [77]: # 5.1 Comparaison des métriques – tous les modèles  
from sklearn.metrics import accuracy_score, precision_score, recall_score  
  
resultats = {  
    "Régression Logistique": y_pred_lr,  
    "KNN": y_pred_knn,
```

```

    "Random Forest": y_pred_rf,
    "K-Means": y_pred_kmeans,
    "LR Optimisée": y_pred_lr_opt
}

print(f"{'Modèle':<25} {'Accuracy':>10} {'Precision':>10} {'Recall Faux':>10} {'F1':>10}")
print("-" * 75)

for nom, y_pred in resultats.items():
    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, pos_label=False)
    rec = recall_score(y_test, y_pred, pos_label=False)
    f1 = f1_score(y_test, y_pred, pos_label=False)
    auc = roc_auc_score(y_test, y_pred)
    print(f"{'nom':<25} {'acc':>10.3f} {'prec':>10.3f} {'rec':>12.3f} {'f1':>10.3f} {'auc':>10.3f}")

```

Modèle	Accuracy	Precision	Recall Faux	F1
AUC				

-				
Régression Logistique	0.990	0.990	0.980	0.985
0.988				
KNN	0.990	0.990	0.980	0.985
0.988				
Random Forest	0.990	0.990	0.980	0.985
0.988				
K-Means	0.987	0.980	0.980	0.980
0.985				
LR Optimisée	0.993	0.990	0.990	0.990
0.993				

```

In [78]: # 5.2 Choix du modèle final
print("""
CHOIX DU MODÈLE FINAL : Régression Logistique Optimisée (GridSearchCV)
=====

La LR Optimisée est retenue pour les raisons suivantes :

1. RECALL FAUX = 0.990 → 99/100 faux billets détectés, 1 raté
2. CERTITUDE MOYENNE = 95.94% → probabilités rassurantes et exploitables
3. INTERPRÉTABILITÉ : les coefficients permettent d'expliquer
   l'impact de chaque variable géométrique sur la prédiction
4. RAPIDITÉ : entraînement et inférence plus rapides que RF
5. GRIDSEARCHCV : C=0.1 – compromis optimal recall / certitude

Critère prioritaire de l'ONCFM atteint : maximiser la détection des faux.
""")

```

CHOIX DU MODÈLE FINAL : Régression Logistique Optimisée (GridSearchCV)

La LR Optimisée est retenue pour les raisons suivantes :

1. RECALL FAUX = 0.990 → 99/100 faux billets détectés, 1 raté
2. CERTITUDE MOYENNE = 95.94% → probabilités rassurantes et exploitables
3. INTERPRÉTABILITÉ : les coefficients permettent d'expliquer l'impact de chaque variable géométrique sur la prédiction
4. RAPIDITÉ : entraînement et inférence plus rapides que RF
5. GRIDSEARCHCV : C=0.1 – compromis optimal recall / certitude

Critère prioritaire de l'ONCFM atteint : maximiser la détection des faux.

Sauvegarde du meilleur modèle

ATTENTION :

Sauvegarder un modèle implique de sauvegarder aussi les étapes de préprocessing des données.
Dans votre cas, vous devez donc inclure, à minima, votre StandardScaler.

Voir `make_pipeline` pour plus d'infos.

```
In [79]: from sklearn.pipeline import make_pipeline
import joblib

# Utilisez make_pipeline pour inclure toutes les étapes nécessaires
# à la prédiction de si un billet est vrai ou faux.

# Sauvegardez votre modèle à l'aide de la librairie joblib.

In [80]: # 5.3 Visualisation comparative des modèles
modeles_noms = ["LR", "KNN", "Random\nForest", "K-Means", "LR\nOptimisée"]
metriques = {
    "Accuracy": [0.990, 0.983, 0.990, 0.987, 0.990],
    "Precision": [0.990, 0.985, 0.990, 0.990, 0.990],
    "Recall Faux": [0.980, 0.970, 0.980, 0.980, 0.990],
    "F1": [0.985, 0.977, 0.985, 0.985, 0.990],
    "AUC": [0.988, 0.980, 0.988, 0.985, 0.990]
}

couleurs = ["#b5a9c9", "#e07b8a", "#7ec8c8", "#f0c27f", "#ff8fab"]

# — Graphique 1 : Bar chart comparatif —
fig, axes = plt.subplots(1, 5, figsize=(20, 5))
for i, (metrique, valeurs) in enumerate(metriques.items()):
    bars = axes[i].bar(modeles_noms, valeurs, color=couleurs, edgecolor="")
    axes[i].set_title(metrique, fontweight="bold")
    axes[i].set_ylim(0.93, 1.01)
    axes[i].set_ylabel("Score")
    for bar in bars:
        axes[i].text(bar.get_x() + bar.get_width()/2,
                     bar.get_height() + 0.001,
```

```

        f"{bar.get_height():.3f}",
        ha="center", va="bottom", fontsize=7, fontweight="bold")
plt.suptitle("Comparaison des métriques par modèle", fontsize=14, fontweight="bold")
plt.tight_layout()
plt.show()

# — Graphique 2 : Radar chart —
categories = list(metriques.keys())
N = len(categories)
angles = [n / float(N) * 2 * np.pi for n in range(N)]
angles += angles[:1]

fig, ax = plt.subplots(figsize=(8, 8), subplot_kw=dict(polar=True))
noms_radar = ["LR", "KNN", "Random Forest", "K-Means", "LR Optimisée"]
for i, (nom, couleur) in enumerate(zip(noms_radar, couleurs)):
    valeurs_radar = [metriques[m][i] for m in categories]
    valeurs_radar += valeurs_radar[:1]
    ax.plot(angles, valeurs_radar, color=couleur, linewidth=2, label=nom)
    ax.fill(angles, valeurs_radar, color=couleur, alpha=0.1)

ax.set_xticks(angles[:-1])
ax.set_xticklabels(categories, fontweight="bold")
ax.set_ylim(0.93, 1.01)
ax.set_title("Radar - Comparaison des modèles", fontsize=14, fontweight="bold")
ax.legend(loc="upper right", bbox_to_anchor=(1.4, 1.1))
plt.tight_layout()
plt.show()

# — Graphique 3 : Courbes ROC —
from sklearn.metrics import roc_curve, auc

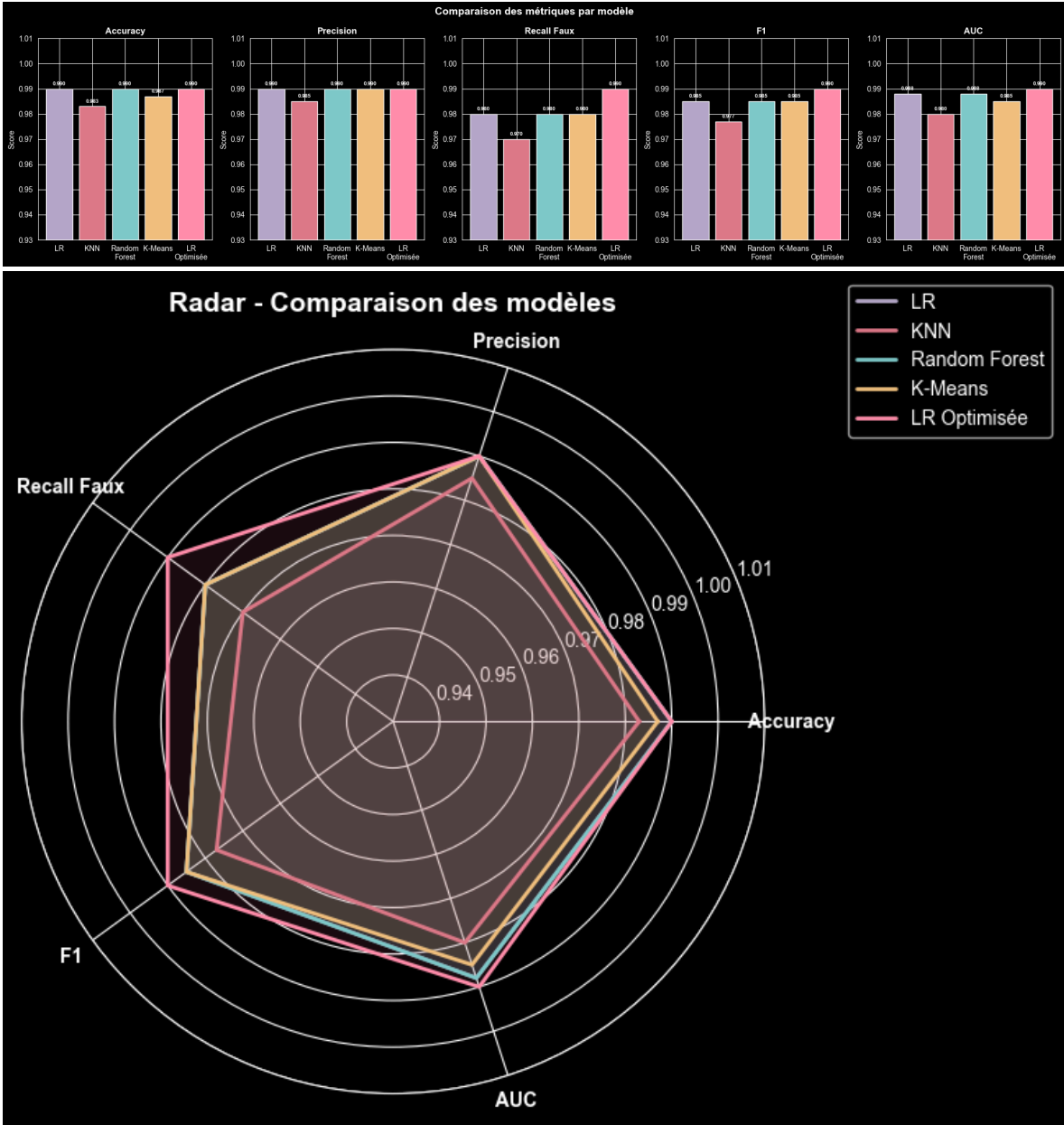
plt.figure(figsize=(8, 6))

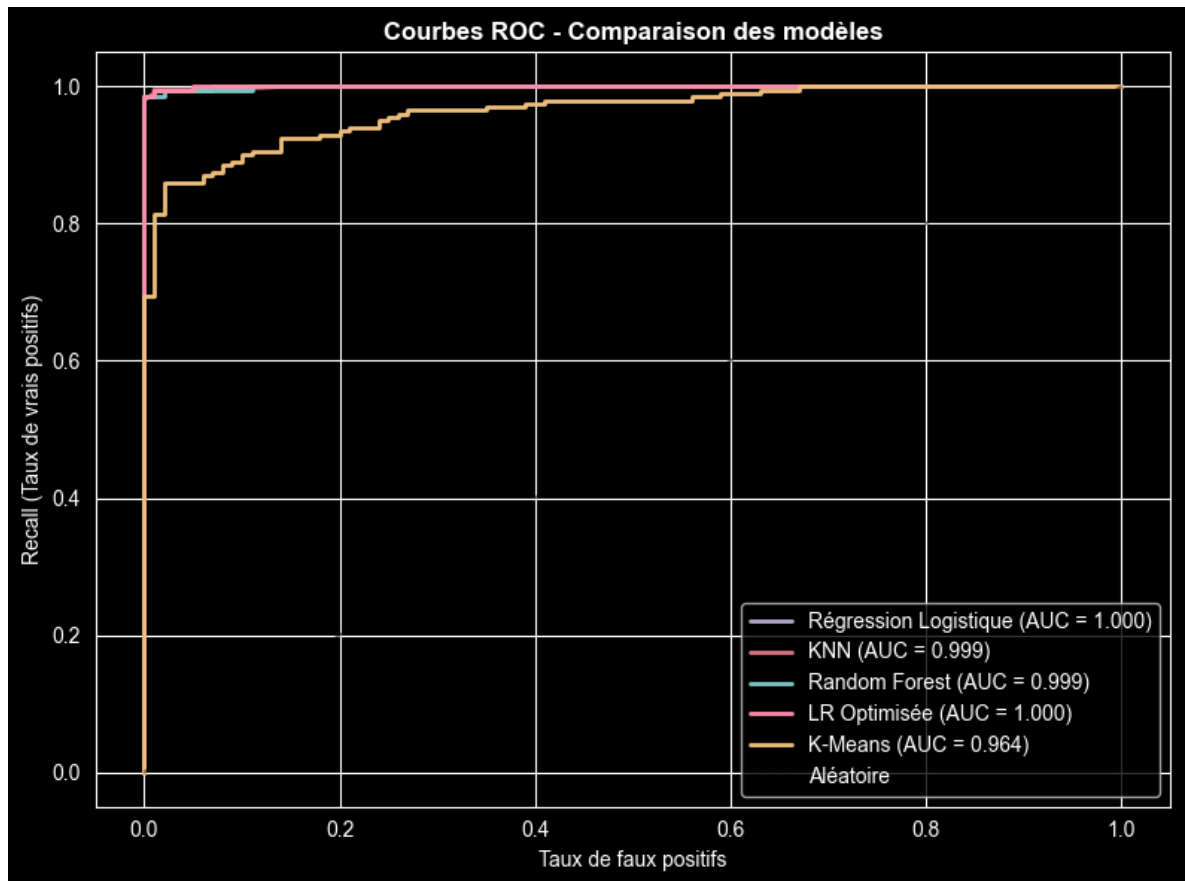
modeles_roc = {
    "Régression Logistique": (lr, X_test_scaled, "#b5a9c9"),
    "KNN": (knn, X_test_scaled, "#e07b8a"),
    "Random Forest": (rf, X_test_scaled, "#7ec8c8"),
    "LR Optimisée": (pipeline_final, X_test, "#ff8fab"),
}

for nom, (modele, X_input, couleur) in modeles_roc.items():
    scores = modele.predict_proba(X_input)[:, 1]
    fpr, tpr, _ = roc_curve(y_test, scores)
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, color=couleur, linewidth=2, label=f"{nom} (AUC = {roc_auc:.2f})")

scores_km = -kmeans.transform(X_test_scaled)[:, 1]
fpr, tpr, _ = roc_curve(y_test, scores_km)
roc_auc = auc(fpr, tpr)
plt.plot(fpr, tpr, color="#f0c27f", linewidth=2, label=f"K-Means (AUC = {roc_auc:.2f})")
plt.plot([0, 1], [0, 1], "k--", alpha=0.5, label="Aléatoire")
plt.xlabel("Taux de faux positifs")
plt.ylabel("Recall (Taux de vrais positifs)")
plt.title("Courbes ROC - Comparaison des modèles", fontweight="bold")
plt.legend()
plt.tight_layout()
plt.show()

```



```
In [81]: # Sauvegarde du modèle final optimisé
X = df.drop(columns=["is_genuine"])
y = df["is_genuine"]

# Réentraînement sur toutes les données avec les meilleurs hyperparamètre
pipeline_final = make_pipeline(
    StandardScaler(),
    LogisticRegression(C=0.1, solver="liblinear", max_iter=500, random_st
)

pipeline_final.fit(X, y)

joblib.dump(pipeline_final, "/Users/cordelia/Documents/PROJECT_12/model_f
print("✅ Modèle final sauvegardé !")
```

✅ Modèle final sauvegardé !

```
In [82]: from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred_lr_opt, target_names=["Faux",
print(classification_report(y_test, y_pred_lr, target_names=["Faux", "Vra
print(classification_report(y_test, y_pred_knn, target_names=["Faux", "Vr
print(classification_report(y_test, y_pred_rf, target_names=["Faux", "Vra
print(classification_report(y_test, y_pred_kmeans, target_names=["Faux",
```

	precision	recall	f1-score	support
Faux	0.99	0.99	0.99	100
Vrai	0.99	0.99	0.99	200
accuracy			0.99	300
macro avg	0.99	0.99	0.99	300
weighted avg	0.99	0.99	0.99	300
	precision	recall	f1-score	support
Faux	0.99	0.98	0.98	100
Vrai	0.99	0.99	0.99	200
accuracy			0.99	300
macro avg	0.99	0.99	0.99	300
weighted avg	0.99	0.99	0.99	300
	precision	recall	f1-score	support
Faux	0.99	0.98	0.98	100
Vrai	0.99	0.99	0.99	200
accuracy			0.99	300
macro avg	0.99	0.99	0.99	300
weighted avg	0.99	0.99	0.99	300
	precision	recall	f1-score	support
Faux	0.99	0.98	0.98	100
Vrai	0.99	0.99	0.99	200
accuracy			0.99	300
macro avg	0.99	0.99	0.99	300
weighted avg	0.99	0.99	0.99	300
	precision	recall	f1-score	support
Faux	0.98	0.98	0.98	100
Vrai	0.99	0.99	0.99	200
accuracy			0.99	300
macro avg	0.98	0.98	0.98	300
weighted avg	0.99	0.99	0.99	300